

Sudarshan Compiler Design Laboratory

Comprehensive Technical Specification & Viva Examination Documentation

Author: Lakshya Singh (Reg No.: 24BDS0054) | Repository:
https://github.com/laksh1357/MyOWN_Compiler

1. Project Overview & Architecture

Sudarshan is a clean, modular imperative programming language implemented in Python 3. The compiler architecture strictly implements all 5 classic compiler pipeline phases: Lexical Analysis, Syntax Analysis (AST), Semantic Analysis (Symbol Table), Three-Address Code (TAC) Generation & Optimization, and Virtual Machine Execution.

Source Code (.mini)	
â	
Phase 1: Lexer (lexer.py)	â â â " Token Stream (Line/Col info)
â	
Phase 2: Parser (parser.py & ast_nodes.py)	â â â " Abstract Syntax Tree (AST)
â	
Phase 3: Semantic Analyzer (semantic.py)	â â â " Scoped Symbol Table
â	
Phase 4: TAC Generator (tac.py & tac_opt.py)	â â â " 3-Address Code Quadruples
â	
Phase 5: Virtual Machine (interpreter.py)	â â â " Execution & Output

Key Technical Innovations

- Multi-Pass Constant Folding & Propagation: Evaluates math & comparison logic at compile time.
- Static Branch Pruning & DCE: Converts constant branches (IF_FALSE 0 -> JUMP) and eliminates dead quadruples.
- Scope Name Mangling: Preserves block scope isolation (x_s1) in flat register TAC memory.
- AST ASCII Visualizer: Formats syntax tree hierarchies into clean ASCII trees without third-party tools.
- Zero-Dependency Handwritten Pipeline: 100% handwritten scanner, parser, symbol table, TAC IR, VM.

2. End-to-End Program Transformation Walkthrough

Sample Sudarshan Program:

```
int x = 5;
while (x < 10) {
    print(x);
    x = x + 1;
}
```

Phase 1: Lexical Analysis (lexer.py)

Scans source string character by character. Converts text into a stream of typed Token objects with precise line and column tracking.

```
[Token(INT, 'int', L1:C1), Token(ID, 'x', L1:C5), Token(ASSIGN, '=', L1:C7),
Token(NUMBER, 5, L1:C9), Token(SEMICOLON, ';', L1:C10),
Token(WHILE, 'while', L2:C1), Token(LPAREN, '(', L2:C7), Token(ID, 'x', L2:C8),
Token(LT, '<', L2:C10), Token(NUMBER, 10, L2:C12), Token(RPAREN, ')', L2:C14),
Token(LBRACE, '{', L2:C16), Token(PRINT, 'print', L3:C5), Token(ID, 'x', L3:C11),
Token(SEMICOLON, ';', L3:C13), Token(ID, 'x', L4:C5), Token(ASSIGN, '=', L4:C7),
Token(ID, 'x', L4:C9), Token(PLUS, '+', L4:C11), Token(NUMBER, 1, L4:C13),
Token(SEMICOLON, ';', L4:C14), Token(RBRACE, '}', L5:C1), Token(EOF, 'EOF', L5:C2)]
```

Phase 2: Syntax Analysis (parser.py)

LL(1) Recursive Descent parser enforcing mathematical operator precedence to construct the hierarchical Abstract Syntax Tree.

```
Program:
  VarDecl (int x):
    IntegerLiteral(5)
  WhileStmt:
    Condition:
      BinaryExpr ('<'):
        Variable(x)
        IntegerLiteral(10)
    Body:
      Block:
        PrintStmt:
          Variable(x)
        Assignment (x =):
          BinaryExpr ('+'):
            Variable(x)
            IntegerLiteral(1)
```

Phase 3: Semantic Analysis (semantic.py & symbol_table.py)

Performs AST Visitor traversal to verify variable declaration before use, prevent duplicate declarations in the same scope, and manage lexical scope hierarchies.

```
=== Scope Level 0 ===
Symbol(x: int, Scope 0 (L1:C1))
```

Phase 4: Intermediate Code Generation (tac.py)

Translates AST nodes into linear 3-Address Code quadruples (op, arg1, arg2, result) with unique temporary registers and jump labels.

```
1:  t1 = 5
2:  x = t1
3:  L1:
4:  t2 = 10
5:  t3 = x LT t2
6:  IF_FALSE t3 GOTO L2
7:  PRINT x
8:  t4 = 1
9:  t5 = x ADD t4
10: x = t5
11: GOTO L1
12: L2:
```

Phase 5: Code Execution / Virtual Machine (interpreter.py)

Executes TAC quadruples on a virtual machine environment with a program counter and memory map.

```
Program Execution Output:
```

```
5  
6  
7  
8  
9
```

```
Execution finished successfully.
```

3. Viva Examination Questions & Answers (20 Key Questions)

Q1: What is lexical analysis?

Lexical analysis is the first compiler phase. It converts raw source characters into meaningful tokens while tracking line/column information and stripping comments/whitespace.

Q2: What is parsing?

Parsing (syntax analysis) takes the token stream from the lexer and verifies whether it conforms to the formal BNF grammar, constructing an Abstract Syntax Tree (AST).

Q3: Why recursive descent parsing?

Recursive descent parsing is a top-down technique where each non-terminal grammar rule is written as a clean, deterministic Python function. It requires zero external tools.

Q4: What is an AST?

An Abstract Syntax Tree (AST) is a hierarchical tree representation of source code logic, omitting syntactic noise like semicolons and parentheses.

Q5: What is semantic analysis?

Semantic analysis checks static rules that context-free grammars cannot capture, such as declaration before use, block scope visibility, and type consistency.

Q6: What is a symbol table?

A symbol table is a data structure storing identifier metadata (name, type, scope level, declaration location) during compilation.

Q7: What is lexical scope?

Lexical scope means variable visibility is determined by physical position in block structures ({ ... }). Child blocks access parent variables, but not vice-versa.

Q8: What is variable shadowing?

Variable shadowing occurs when an inner block declares a variable with the same name as an outer variable, temporarily hiding the outer declaration.

Q9: What is Three-Address Code (TAC)?

TAC is an intermediate representation where each quadruple has at most 3 operands, breaking complex logic into flat, register-like hardware steps.

Q10: Why use temporary variables in TAC?

Temporary variables (t1, t2, ...) break down nested AST expressions into linear step-by-step instructions.

Q11: What is constant folding?

Constant folding evaluates constant expressions (e.g. $2 + 3 * 4 \rightarrow 14$) at compile time to save runtime computation.

Q12: What is dead code elimination?

Dead code elimination removes unreachable instructions (e.g. quadruples following an unconditional GOTO jump).

Q13: Why interpret TAC instead of AST directly?

Interpreting TAC proves that intermediate code generation works correctly and demonstrates a complete 5-phase compiler pipeline.

Q14: Difference between syntax and semantic errors?

Syntax errors violate grammar rules (e.g. missing semicolon). Semantic errors violate static logic rules (e.g. using an undeclared variable).

Q15: Difference between compile-time and runtime errors?

Compile-time errors (lexical, syntax, semantic) are caught before execution. Runtime errors (e.g. division by zero) occur during VM execution.

Q16: How are while loops represented in TAC?

Using two labels and jumps: a start label L1, condition evaluation, IF_FALSE GOTO L2, body quadruples, GOTO L1, and end label L2.

Q17: How is if/else represented in TAC?

Condition evaluation, IF_FALSE GOTO L1 (else), then-branch quadruples, GOTO L2 (end), label L1, else-branch quadruples, label L2.

Q18: How is division by zero handled?

Skipped during compile-time constant folding to avoid compiler crashes; caught during VM execution by raising a clean RuntimeError.

Q19: How is operator precedence implemented?

Through grammar method hierarchy: comparison -> addExpr (+/-) -> term (*/) -> factor, ensuring higher precedence operators bind deeper in AST.

Q20: What are the main limitations of Sudarshan?

Sudarshan supports only integer variables and single-file programs without functions, arrays, floats, or target assembly code generation.